

Lab4-1 : lab-exmem-fir : Executing FIR function from User Project memory

Student Name: Yip Hoi Shun

Student ID: 21039456

List relevant code changes from rtl/firmware

firmware

fir.c

function initfir() contain a for loop which reset the input buffer and output signal array to 0

function fir() contain a two layer for loop in which would calculate the FIR result by multiplying the inputbuffer to the taps

```
#include "fir.h"

void __attribute__((section(".mprjram"))) initfir()
{
    // initial your fir
    for (int i = 0; i < N; i++)
    {
        inputbuffer[i] = 0;
        outputsignal[i] = 0;
    }
}

int *__attribute__((section(".mprjram"))) fir()
{
    initfir();

    // write down your fir
    for (int i = 0; i < N; i++)
    {
        int data_get = inputsignal[i]; // get data from axi-stream
        inputbuffer[i] = data_get;      // store data to bram
        for (int j = 0; j <= i; j++)
        {
            outputsignal[i] += inputbuffer[j] * taps[i - j];
        }
    }
    return outputsignal;
}
```

RTL

user_project_example.counter.v

The wishbone bus controller has a counter acknowledging every 10 cycle when the wishbone chip select *wbs_stb_i* & *wbs_cyc_i* is both raised.

```
wire clk;
wire rst;
assign clk = wb_clk_i;
assign rst = wb_rst_i;

// input to ram
wire ram_en;
wire [3:0] ram_we;
wire [31:0] ram_adr;
wire [31:0] ram_data;
wire enable;

assign enable = wbs_cyc_i & wbs_stb_i;
assign ram_adr = {32{enable}} & wbs_adr_i;
assign ram_data = {32{enable}} & wbs_dat_i ;
assign ram_we = {4 {enable}} & {4{wbs_we_i}} & wbs_sel_i ;
assign ram_en = enable & wbs_sel_i;

reg [3:0] counter;

always @ (posedge clk) begin
    if(rst)begin
        counter <= 4'b0;
    end else begin
        if (enable)begin
            if(counter != DELAYS)begin
                counter <= counter +4'b1;
            end else begin
                counter <= 32'b0;
            end
        end else begin
            counter <=32'b0;
        end
    end
end

assign wbs_ack_o = counter == DELAYS;
bram user_bram (
    .CLK(clk),
    .WE0(ram_we),
    .EN0(ram_en),
    .Di0(wbs_dat_i),
    .Do0(wbs_dat_o),
    .A0(ram_adr)
);
```

bram.v

a parameter of 12 is selected for the BRAM

```
module bram(
    CLK,
    WE0,
    EN0,
    Di0,
    Do0,
    A0
);

    input  wire          CLK;
    input  wire    [3:0]  WE0;
    input  wire          EN0;
    input  wire    [31:0] Di0;
    output reg    [31:0] Do0;
    input  wire    [31:0] A0;

    // 16KB      (// 4 kB)
    // 32 bit = 4 byte -> 16KB = 4 byte * 2 ^ N
    parameter N = 12;
    (* ram_style = "block" *) reg [31:0] RAM[0:2**N-1];
    // 4 MB
    //This RAM has Data width of 8 bits and 4MB address locations
    //reg [7:0] RAM[0:4*1024*1024-1];

    always @(posedge CLK)
        if(EN0) begin
            Do0 <= RAM[A0[N-1:0]];
            if(WE0[0]) RAM[A0[N-1:0]][7:0] <= Di0[7:0];
            if(WE0[1]) RAM[A0[N-1:0]][15:8] <= Di0[15:8];
            if(WE0[2]) RAM[A0[N-1:0]][23:16] <= Di0[23:16];
            if(WE0[3]) RAM[A0[N-1:0]][31:24] <= Di0[31:24];
        end
        else
            Do0 <= 32'b0;

endmodule
```

Memory map & linker (lds)

The memory mapping can be viewed in firmware/sections.lds

```
/* INCLUDE ../../generated/regions.ld */

MEMORY {
    vexriscv debug : ORIGIN = 0xf00f0000, LENGTH = 0x00000100
    dff : ORIGIN = 0x00000000, LENGTH = 0x00000400
    dff2 : ORIGIN = 0x00000400, LENGTH = 0x00000200
    flash : ORIGIN = 0x10000000, LENGTH = 0x01000000
    mprj : ORIGIN = 0x30000000, LENGTH = 0x00100000
    mprjram : ORIGIN = 0x38000000, LENGTH = 0x00400000
    hk : ORIGIN = 0x26000000, LENGTH = 0x00100000
    csr : ORIGIN = 0xf0000000, LENGTH = 0x00010000
}
```

BRAM, which used for FIR calculation is mapped to mpjram. Its address is at 0x38000000, and size is 4 KB.

From counter_la_fir.elf, 0x38000000 is accessed by the fir function

```
3800008c <fir>:
3800008c: fe010113      addi    sp,sp,-32
38000090: 00112e23      sw      ra,28(sp)
38000094: 00812c23      sw      s0,24(sp)
38000098: 00912a23      sw      s1,20(sp)
3800009c: 02010413      addi    s0,sp,32
380000a0: f85ff0ef      jal     ra,38000024 <initfir>
380000a4: fe042623      sw      zero,-20(s0)
380000a8: 0cc0006f      j       38000174 <fir+0xe8>
380000ac: 02c00713      li      a4,44
380000b0: fec42783      lw      a5,-20(s0)
380000b4: 00279793      slli    a5,a5,0x2
380000b8: 00f707b3      add     a5,a4,a5
380000bc: 0007a783      lw      a5,0(a5)
380000c0: fef42223      sw      a5,-28(s0)
380000c4: 05c00713      li      a4,92
380000c8: fec42783      lw      a5,-20(s0)
380000cc: 00279793      slli    a5,a5,0x2
380000d0: 00f707b3      add     a5,a4,a5
380000d4: fe442703      lw      a4,-28(s0)
380000d8: 00e7a023      sw      a4,0(a5)
380000dc: fe042423      sw      zero,-24(s0)
380000e0: 07c0006f      j       3800015c <fir+0xd0>
380000e4: 08800713      li      a4,136
380000e8: fec42783      lw      a5,-20(s0)
380000ec: 00279793      slli    a5,a5,0x2
380000f0: 00f707b3      add     a5,a4,a5
380000f4: 0007a483      lw      s1,0(a5)
380000f8: 05c00713      li      a4,92
380000fc: fe842783      lw      a5,-24(s0)
38000100: 00279793      slli    a5,a5,0x2
38000104: 00f707b3      add     a5,a4,a5
38000108: 0007a683      lw      a3,0(a5)
3800010c: fec42703      lw      a4,-20(s0)
38000110: fe842783      lw      a5,-24(s0)
38000114: 40f707b3      sub     a5,a4,a5
38000118: 00000713      li      a4,0
3800011c: 00279793      slli    a5,a5,0x2
38000120: 00f707b3      add     a5,a4,a5
38000124: 0007a783      lw      a5,0(a5)
38000128: 00078593      mv      a1,a5
3800012c: 00068513      mv      a0,a3
38000130: ed1ff0ef      jal     ra,38000000 <__mulsi3>
38000134: 00050793      mv      a5,a0
38000138: 00f48733      add     a4,s1,a5
```

How to move code from spiflash to user project area memory

The program is compiled to a hex file, named counter_la_fir.hex. Through \$readmemh, the file is loaded in the spiflash.

```

spiflash #(
    .FILENAME("counter_la_fir.hex")
) spiflash (
    .csb(flash_csb),
    .clk(flash_clk),
    .io0(flash_io0),
    .io1(flash_io1),
    .io2(), // not us
    .io3() // not used
);

```

```

initial begin
    $display("Reading %s", FILENAME);
    $readmemh(FILENAME, memory);
    //$display("Memory 5 bytes = 0x%02x 0x%02x 0x%02x 0x%02x 0x%02x",
    //      memory[0], memory[1], memory[2],
    //      memory[3], memory[4]);
    $display("%s loaded into memory", FILENAME);
    $display("Memory 5 bytes = 0x%02x 0x%02x 0x%02x 0x%02x 0x%02x",
        memory[0], memory[1], memory[2],
        memory[3], memory[4]);
end

```

And then read to the soc through the QSPI Flash interface

```

caravel uut (
    .vddio      (VDD3V3),
    .vddio_2    (VDD3V3),
    .vssio      (VSS),
    .vssio_2    (VSS),
    .vdda       (VDD3V3),
    .vssa       (VSS),
    .vccd       (VDD1V8),
    .vssd       (VSS),
    .vdda1      (VDD3V3),
    .vdda1_2    (VDD3V3),
    .vdda2      (VDD3V3),
    .vssa1      (VSS),
    .vssa1_2    (VSS),
    .vssa2      (VSS),
    .vccd1      (VDD1V8),
    .vccd2      (VDD1V8),
    .vssd1      (VSS),
    .vssd2      (VSS),

    .clock      (clock),
    .gpio       (gpio),
    .mprj_io    (mprj_io),
    .flash_csb  (flash_csb),
    .flash_clk  (flash_clk),
    .flash_io0  (flash_io0),
    .flash_io1  (flash_io1),
    .resetb     (RSTB)
);

spiflash #(
    .FILENAME("counter_la_fir.hex")
) spiflash (
    .csb(flash_csb),
    .clk(flash_clk),
    .io0(flash_io0),
    .io1(flash_io1),
    .io2(), // not used
    .io3() // not used
);

// Testbench UART
tbuart tbuart (

```

How to execute code from user project memory

As per previously mention, the fir program is mapped to the mprjram.

disassembly of section `.mprjram:`

```

8000000 <__mulsi3>:
8000000: 00050613      mv      a2,a0
8000004: 00000513      li      a0,0
8000008: 0015f693      andi     a3,a1,1
800000c: 00068463      beqz    a3,38000014 <__mulsi3+0x14>
8000010: 00c50533      add     a0,a0,a2
8000014: 0015d593      srli    a1,a1,0x1
8000018: 00161613      slli    a2,a2,0x1
800001c: fe0596e3      bnez    a1,38000008 <__mulsi3+0x8>
8000020: 00008067      ret

8000024 <initfir>:
8000024: fe010113      addi    sp,sp,-32
8000028: 00812e23      sw      s0,28(sp)
800002c: 02010413      addi    s0,sp,32
8000030: fe042623      sw      zero,-20(s0)
8000034: 0380006f      j       3800006c <initfir+0x48>
8000038: 05c00713      li      a4,92
800003c: fec42783      lw      a5,-20(s0)
8000040: 00279793      slli    a5,a5,0x2
8000044: 00f707b3      add     a5,a4,a5
8000048: 0007a023      sw      zero,0(a5)
800004c: 08800713      li      a4,136
8000050: fec42783      lw      a5,-20(s0)
8000054: 00279793      slli    a5,a5,0x2
8000058: 00f707b3      add     a5,a4,a5
800005c: 0007a023      sw      zero,0(a5)
8000060: fec42783      lw      a5,-20(s0)
8000064: 00178793      addi    a5,a5,1
8000068: fef42623      sw      a5,-20(s0)
800006c: fec42703      lw      a4,-20(s0)
8000070: 00a00793      li      a5,10
8000074: fce7d2e3      bge     a5,a4,38000038 <initfir+0x14>
8000078: 00000013      nop
800007c: 00000013      nop
8000080: 01c12403      lw      s0,28(sp)
8000084: 02010113      addi    sp,sp,32
8000088: 00008067      ret

800008c <fir>:
800008c: fe010113      addi    sp,sp,-32
8000090: 00112e23      sw      ra,28(sp)
8000094: 00812c23      sw      s0,24(sp)
8000098: 00912a23      sw      s1,20(sp)
800009c: 02010413      addi    s0,sp,32
80000a0: f85ff0ef      jal     ra,38000024 <initfir>
80000a4: fe042623      sw      zero,-20(s0)
80000a8: 0cc0006f      j       38000174 <fir+0xe8>
80000ac: 02c00713      li      a4,44
80000b0: fec42783      lw      a5,-20(s0)
80000b4: 00279793      slli    a5,a5,0x2
80000b8: 00f707b3      add     a5,a4,a5
80000bc: 0007a783      lw      a5,0(a5)
80000c0: fef42223      sw      a5,-28(s0)
80000c4: 05c00713      li      a4,92
80000c8: fec42783      lw      a5,-20(s0)
80000cc: 00279793      slli    a5,a5,0x2
80000d0: 00f707b3      add     a5,a4,a5
80000d4: fe442703      lw      a4,-28(s0)
80000d8: 00e7a023      sw      a4,0(a5)
80000dc: fe042423      sw      zero,-24(s0)
80000e0: 07c0006f      j       3800015c <fir+0xd0>
80000e4: 08800713      li      a4,136
80000e8: fec42783      lw      a5,-20(s0)
80000ec: 00279793      slli    a5,a5,0x2
80000f0: 00f707b3      add     a5,a4,a5
80000f4: 0007a483      lw      s1,0(a5)
80000f8: 05c00713      li      a4,92
80000fc: fe842783      lw      a5,-24(s0)
8000100: aa279793      slli    a5,a5,0x2

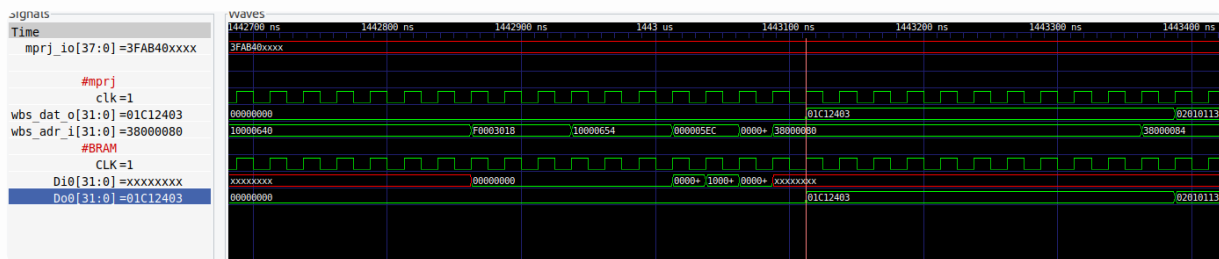
```

```

8000100:      00279793      slli      a5,a5,0x2
8000104:      00f707b3      add       a5,a4,a5
8000108:      0007a683      lw        a3,0(a5)
800010c:      fec42703      lw        a4,-20(s0)
8000110:      fe842783      lw        a5,-24(s0)
8000114:      40f707b3      sub       a5,a4,a5
8000118:      00000713      li        a4,0
800011c:      00279793      slli      a5,a5,0x2
8000120:      00f707b3      add       a5,a4,a5
8000124:      0007a783      lw        a5,0(a5)
8000128:      00078593      mv        a1,a5
800012c:      00068513      mv        a0,a3
8000130:      ed1ff0ef      jal       ra,38000000 <__mulsi3>
8000134:      00050793      mv        a5,a0
8000138:      00f48733      add       a4,s1,a5
800013c:      08800693      li        a3,136
8000140:      fec42783      lw        a5,-20(s0)
8000144:      00279793      slli      a5,a5,0x2
8000148:      00f687b3      add       a5,a3,a5
800014c:      00e7a023      sw        a4,0(a5)
8000150:      fe842783      lw        a5,-24(s0)
8000154:      00178793      addi      a5,a5,1
8000158:      fef42423      sw        a5,-24(s0)

```

When fir function is called, the code is executed from the mprjram



Show the Operation sequence and its waveform

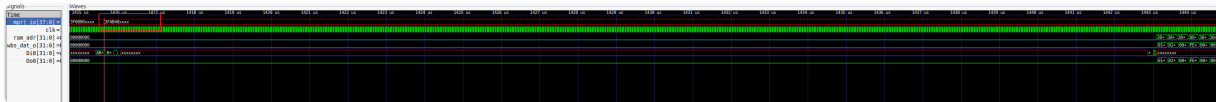
When the mprj_io is set to 0xAB40 the FIR computation started

```

// Flag start of the test
reg_mprj_data1 = 0xAB400000;

```


anager

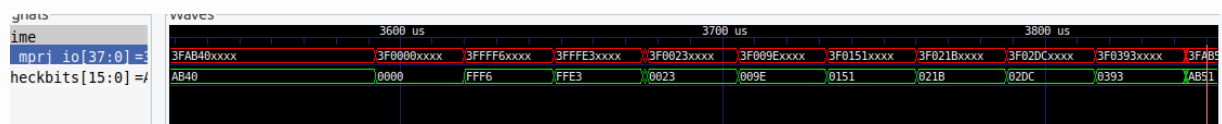


```

12 }
13 int *__attribute__((section(".mprjram"))) fir()
14 {
15     initfir();
16
17     // write down your fir
18     for (int i = 0; i < N; i++)
19     {
20         int data_get = inputsignal[i]; // get data from axi-stream
21         inputbuffer[i] = data_get;      // store data to bram
22         for (int j = 0; j <= i; j++)
23         {
24             outputsignal[i] += inputbuffer[j] * taps[i - j];
25         }
26     }
27     return outputsignal;
28 }

```

```
//nrint("\n").
```



When it reached AB51 which set by counter_la_fir.c, the test finished

```
141         reg_mprj_data1 = ~(cmp+10) << 10;
142
143         //print("\n");
144         //print("Monitor: Test 1 Passed\n\n");           // Makes simulation very long!
145         reg_mprj_data1 = 0xAB510000;
146     }
147
```

Time	3F020Cxxxx	3F0393xxxx	3F0451xxxx
mpri_io[37:0]=3F044Axxxx			
checkbits[15:0]=044A	020C	0393	AB51

Simulation Result

```
ubuntu@ubuntu2004:~/Desktop/Final/SOC_LAB4_1-main/testbench/counter_la_fir$ ./run_sim
Reading counter_la_fir.hex
counter_la_fir.hex loaded into memory
Memory 5 bytes = 0x6f 0x00 0x00 0x0b 0x13
VCD info: dumpfile counter_la_fir.vcd opened for output.
LA Test 1 started
LA Test 2 passed
ubuntu@ubuntu2004:~/Desktop/Final/SOC_LAB4_1-main/testbench/counter_la_fir$
```